

MC 302 – DBMS: Relational Algebra

Dr. Goonjan Jain

Department of Applied Mathematics

Delhi Technological University

Relational Algebra

- Basic set of operations for the formal relational model.
- Operations enable a user to specify basic retrieval requests
- Result of a query is a new relation
- Sequence of operations forms a relational algebra expression
- Relational algebra is a formal system for manipulating and querying relational databases.
- It provides a set of operations that can be applied to relations (tables) to retrieve, filter, and combine data.
- Relational algebra serves as the theoretical foundation for database query languages, and it enables users to express complex operations on relational data in a systematic way.
- relational algebra is a formal, mathematical notation used for theoretical purposes.
- In practical database systems, query languages like SQL are used to interact with relational databases, and these languages are based on the principles of relational algebra.
- The operations in relational algebra provide a foundation for understanding and optimizing queries in relational databases.

Unary Relational Operator – SELECT (σ)

- Used to choose a subset of tuples(rows) from a relation, satisfying the selection condition.

$$\sigma_{\langle \text{selection condition} \rangle}(R)$$

- E.g. To select the employee tuple whose department is 4.

$$\sigma_{\text{dno} = 4}(\text{Employee})$$

- Selection condition is a Boolean expression.
- E.g. select tuples of all employees who either work in department 4 and make over 45,000/week or work in department 5 and make over 30,000/week

$$\sigma_{(\text{dno} = 4 \text{ AND } \text{sal} > 25000) \text{ OR } (\text{dno} = 5 \text{ AND } \text{sal} > 30000)}(\text{Employee})$$

- Operation is commutative

$$\sigma_{\langle \text{cond 1} \rangle}(\sigma_{\langle \text{cond 2} \rangle}(R)) = \sigma_{\langle \text{cond 2} \rangle}(\sigma_{\langle \text{cond 1} \rangle}(R))$$

- In SQL, SELECT condition is specified in the WHERE clause of a query.

Unary Relation Operation – PROJECT (π)

- The projection operation retrieves specific columns (attributes) from a relation, discarding the others.
- Example: $\pi_{\text{Name, Age}}(\text{Student})$
- Removes any duplicate tuples – *Duplicate Elimination*
- If duplicates not eliminated – result set would be a multiset or bag of tuples rather than a set – not permitted in formal relational model.
- In SQL, PROJECT attribute list is specified in the SELECT clause of a query

Union (U)

- The union operation combines tuples from two relations, removing duplicates.
- Example: $R \cup S$
- R and S should be *union compatible*
 - Same number of columns
 - $\text{Domain}(R_i) = \text{Domain}(S_i), 1 \leq i \leq n$

Intersection ($\cap$)

- The intersection operation returns common tuples present in both relations.
- Example: $R \cap S$

Set Difference (-)

- The set difference operation returns tuples that are in the first relation but not in the second.
- Example: $R - S$

Cartesian Product ($\times$)

- The Cartesian product operation combines every tuple from the first relation with every tuple from the second.
- It results in a new relation where each tuple is a combination of a tuple from the first relation and a tuple from the second relation
- Example: $R \times S$
- The resulting relation has a number of attributes equal to the sum of the attributes in R and S . If R has n attributes and S has m attributes, the result will have $n+m$ attributes.
- The resulting tuples combine all possible combinations of tuples from R and S .

Rename ()

- The renaming operation is used to assign new names to relations or attributes.
- It allows you to change the names of either entire relations or specific attributes within a relation.

$$\rho_{NewName}(Students)$$

- This operation renames the relation *Students* to *NewName*.

$$\rho_{NewID(StudentID),NewName(Name)}(Students)$$

- This operation renames the attributes *StudentID* and *Name* in the relation *Students* to *NewID* and *NewName*, respectively.

Join ($\bowtie$)

- The join operation combines tuples from two relations based on a specified condition.
- The result of a join is a new relation that includes all possible combinations of tuples from the two input relations that satisfy the given condition.

$$R \bowtie_{\text{condition}} S$$

- *Inner join*
- *Natural Join*
- *Theta Join*
- *Outer Join*

Inner join

- an inner join is a type of join operation that combines rows from two or more tables based on a related column between them.
- The result of an inner join includes only the rows that have matching values in the specified column or columns.
- Inner join is one of the most commonly used join operations.

Students ⋈ *Grades*
Students.StudentID=Grades.StudentID

```
SELECT columns
FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

```
SELECT Employees.EmployeeName, Departments.DepartmentName
FROM Employees
INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

Natural Join

- the natural join is an operation that combines tuples from two relations based on common attributes with the same name and data type
- the result contains all tuples from the cross product $R \times S$ where the values in the common attributes match.
- It's important to note that the natural join operation automatically identifies the common attributes based on their names and data types.
- While natural join can be convenient, it also has potential drawbacks, such as unexpected results when schema changes occur or when there are multiple attributes with the same name but different meanings.

```
SELECT EmployeeName, DepartmentName  
FROM Employees  
NATURAL JOIN Departments;
```

Natural Join

- Consider two tables, Employees and Departments, both having a column named DepartmentID.
- To retrieve the names of employees and their corresponding departments using a natural join, the SQL query would be:

```
SELECT EmployeeName, DepartmentName  
FROM Employees  
NATURAL JOIN Departments;
```

- In this example, the natural join operation automatically identifies the common column DepartmentID and performs the join based on it.
- The result would include columns from both tables where the DepartmentID values match.

Theta Join ($\bowtie_{\theta}$)

- a theta join is a type of join operation that combines rows from two tables based on a condition specified using a theta (θ) or comparison operator.

$$R \bowtie_{\theta} S$$

- d

Theta Join

- Consider two tables, Employees and Departments, with a common column DepartmentID. To retrieve the names of employees and their corresponding departments where the employee's salary is greater than a certain threshold, you might use a theta join:

```
SELECT Employees.EmployeeName, Departments.DepartmentName  
FROM Employees  
JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID  
WHERE Employees.Salary > 50000;
```

- In this example, the ON clause specifies the condition for joining based on the common attribute DepartmentID, and the WHERE clause further filters the result based on the salary condition.

Outer Join

- an outer join is a type of join operation that returns unmatched rows from one or both tables, in addition to the matched rows.
- Outer joins are useful when you want to include rows from one table even if there is no corresponding match in the other table.
 - Left outer Join
 - Right Outer Join
 - Full Outer Join

Left Outer Join

- Returns all rows from the left table and the matching rows from the right table. If there is no match in the right table, NULL values are returned for the columns from the right table.
- Syntax (SQL): `SELECT * FROM table1 LEFT JOIN table2 ON condition;`

```
SELECT Employees.EmployeeID, Employees.EmployeeName, Departments.DepartmentName  
FROM Employees  
LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

Right Outer Join

- Returns all rows from the right table and the matching rows from the left table. If there is no match in the left table, NULL values are returned for the columns from the left table.

```
SELECT Employees.EmployeeID, Employees.EmployeeName, Departments.DepartmentName  
FROM Employees  
RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

Full Outer Join

- Returns all rows when there is a match in either the left or the right table. If there is no match in either table, NULL values are returned for the columns from the table without a match.

```
SELECT Employees.EmployeeID, Employees.EmployeeName, Departments.DepartmentName  
FROM Employees  
FULL JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```